

Deriving Tarjan's SCC Algorithm From Scratch

A discovery-order derivation of low-link • the full struggle over the three edge types and dfn-vs-low • correctness proof • bug-by-bug post-mortem • the Turing Award & first principles

Technical notes • jsjhoubo.github.io

Contents

- 0 • Memorizing an algorithm vs. owning one
- 1 • Pinning the problem: digraphs, reachability, equivalence, SCC
- 2 • The naive $O(V^4)$ solution, and the direction it points to
- 3 • Forcing out low: a quantity that is needed, not defined
- 4 • The three-edge struggle (core part I): which info is usable
- 5 • `dfn[j]` or `low[j]`: the most subtle fork
- 6 • The root test and the stack: extracting a component cleanly
- 7 • Correctness: chaining every judgment with no gaps
- 8 • The gap from proof to code (core part II): bug-by-bug
- 9 • Full code + seven test cases
- 10 • Where it came from: the Turing Award and the 1972 paper
- 11 • Coda: Euler's bridges, modeling > solving, first principles, manuscripts

0 • Memorizing an algorithm vs. owning one

Almost every Tarjan tutorial looks the same: drop the definitions of `dfn` and `low`, tell you "for the three edge types, update low like so," paste the code. You memorize it, you pass the problem — but the moment an interviewer probes from another angle ("why does this edge use `dfn[j]` while that one uses `low[j]`?", "why is low initialized that way?") the memorized layer collapses, because you hold only the conclusions, not the chain of reasoning that produced them.

This note does the opposite: **it does not presuppose what low is. It starts from the most primitive question and forces out low, the closure test, and the stack, one step at a time.** More importantly, it **preserves the real detours, misjudgments, and self-rebuttals along the way** — because those "I thought X, turned out wrong, because Y, so changed to Z" struggles are exactly what lets you own the algorithm rather than merely recall it.

This article is full of "Q / A / FIX" dialogue blocks. They are not decoration — they are the deliberately preserved **scene of thinking**: Q = where I got stuck or grew suspicious, A = the move at that moment, FIX = later realizing the earlier step was wrong and how it was corrected. When you read, pause at each Q and think for yourself before reading on — that is the right way to follow a forward derivation.

1 • Pinning the problem: digraphs, reachability, equivalence, SCC

Every derivation needs an immovable foundation. Our object — the "strongly connected component" — must first have a description that depends on no algorithm, purely on definitions. Every later "why is this step correct" must trace back to this section.

1.1 Reachability

Directed graph $G = (V, E)$, V the vertex set, $E \subseteq V \times V$ the directed edges. We say u **reaches** v (written $u \rightsquigarrow v$) iff there is a directed path $u = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k = v$, with $k \geq 0$ and each $(x_i, x_{i+1}) \in E$. In particular, for $k=0$, $u \rightsquigarrow u$ always holds — **every vertex reaches itself in zero steps**. This "zero step" looks trivial; it turns out to be the root of "a single vertex is a valid SCC."

1.2 Mutual reachability, and that it is an equivalence relation

Define on V : $u \sim v \Leftrightarrow (u \rightsquigarrow v) \text{ and } (v \rightsquigarrow u)$, i.e. the two can reach each other. We verify, point by point, that it is an **equivalence relation**, because this is what makes "SCCs form a clean partition":

- **Reflexive:** $u \sim u$, immediate from zero-step reachability.
- **Symmetric:** the two conjuncts are symmetric in u and v .
- **Transitive:** if $u \sim v$ and $v \sim w$, then $u \rightsquigarrow v \rightsquigarrow w$ so $u \rightsquigarrow w$; conversely $w \rightsquigarrow v \rightsquigarrow u$ so $w \rightsquigarrow u$.

Hence $u \sim w$.

1.3 An SCC = an equivalence class

Since $\sim$ is an equivalence relation, it cuts V into **disjoint classes whose union is V** . Each class is a **strongly connected component (SCC)**: $[u] = \{ v \in V \mid u \sim v \}$. Equivalently: an SCC is a **maximal** subset $C \subseteq V$ in which every pair is mutually reachable — "maximal" meaning **adding any further vertex would break the all-pairs mutual reachability**.

This word "maximal" will trigger a genuine argument later. Someone asks: in a set of four mutually-reachable vertices, take a three-vertex subset — aren't those three still mutually reachable in the original graph? Yes — but that three-vertex set is a **strongly connected subgraph**, not a **strongly connected component**; the word "component" **itself contains maximality**. Subgraphs can nest; components (equivalence classes) cannot. We will need this distinction when proving "components do not nest."

1.4 A corollary you can squeeze out immediately: the condensation is a DAG

Since $\sim$ partitions V into disjoint classes, reachability between two distinct SCCs is **necessarily one-directional**: you cannot have both $C_1 \rightsquigarrow C_2$ and $C_2 \rightsquigarrow C_1$ — otherwise their vertices would be mutually reachable and should merge into one class, contradicting that they are distinct. Collapse each SCC to a point, and the inter-component edges form a **directed acyclic graph (DAG)**. We will not use this corollary directly, but it is the fundamental reason SCCs are worth isolating: they let any digraph be compressed into a DAG.

2 • The naive $O(V^4)$ solution, and the direction it points to

Do not chase cleverness up front. Write the most straightforward, certainly-correct, but slow solution first — it often **tells you where the bottleneck is**, and the bottleneck is the entrance to optimization.

2.1 Straight from the definition

To test whether u and v share an SCC, follow 1.2: DFS/BFS from u to see if it reaches v , then from v to see if it returns to u ; both succeed $\Rightarrow u \sim v$. One traversal is $O(V+E)$. For all $C(V,2) \approx V^2$ pairs, run it once each, union the mutually-reachable pairs with a DSU, and each resulting block is an SCC.

Total: V^2 pairs $\times O(V+E)$ each = $O(V^2(V+E))$, roughly $O(V^4)$ on dense graphs. The DSU itself is near $O(V\alpha(V))$, negligible; the bottleneck is entirely those V^2 traversals.

2.2 Where it wastes work — this is the point

Absurdly slow — but it **exposes the exact location of the waste**: the single DFS from u already **knows every vertex u can reach** — a whole sheet of information. Yet you use it to answer one yes/no ("can u reach this particular v "), then throw the sheet away. The next pair starts over. The same reachability is recomputed on the order of $O(V)$ times.

The key is in this sentence: one DFS contains far more than "can u reach v ." If we can design a quantity that lets **a single DFS, while backtracking, compute "who shares an SCC with whom" as a by-product**, we compress V^2 passes into a **constant number of linear DFS passes**, achieving $O(V+E)$. That is Tarjan's goal, and the **motivation** for the quantity low. Note: low does not fall from the sky as a definition — it is the product forced out by the need "to determine SCCs within one DFS."

3 • Forcing out low: a quantity that is needed, not defined

Now the real derivation begins. Instead of writing "the definition of low is...", we ask: **to judge component boundaries within one DFS, what exactly do I need to know?**

3.1 The DFS tree, and the core question

A DFS on a digraph forms a DFS tree along the "first-visit" edges (the rest are back / forward / cross edges). When the DFS **backtracks** from a vertex u (all of u 's out-edges processed, about to return), I want to decide, in place:

Does the DFS subtree rooted at u exactly **"cap off" an SCC?**

— Equivalently: does any vertex in u 's subtree **"climb back to u , or above u (an earlier-visited ancestor)"**?

Q► Why is "can anything climb above u " equivalent to "is u the top of a component"?

A► Because strong connectivity demands mutual reachability. Vertices in u 's subtree are reachable from u via tree edges (u is above them); whether they can get back to u or u 's ancestors decides whether they and u (and u 's ancestors) are **mutually** reachable, hence in one SCC.

- If **nothing** in the subtree can climb above $u \Rightarrow$ this cluster cannot escape, cannot return to a higher ancestor, so u is the highest point of the cluster: u caps it, the component ends at u .

- If **something** can climb to an ancestor $a \Rightarrow$ then u , a , and the climbing path form a larger cycle whose vertices are mutually reachable; u cannot be the top, the real top is at a 's level or higher.

So the core piece of information surfaces: from u 's subtree, **how far back can we "reach up" — what is the earliest vertex?** If I maintain, for each u , this "earliest reachable vertex," I can compare it to u 's own position and decide whether u caps. **Call this "earliest vertex reachable from u and its subtree" $\text{low}[u]$.** Note: low is **forced** out by the need to "judge capping," not defined first and then used.

3.2 First precision question: store a "node" or an "index"?

Q► low[u] should hold "that earliest vertex" — hold what exactly? The node itself (its id)?

A► No need. We only care about "who is earlier," a **comparison**; it needs no identity, only a comparable scalar. So stamp each vertex, on **first visit**, with an increasing **discovery number** dfn[u]. low[u] stores that number — "how far back" reduces to "smallest dfn." The node id never enters low (it is useful only later, sitting in the stack to extract members).

3.3 Second precision question: "discovery order" or "depth"? — an easy trap

Q► Could that index be "DFS depth" (levels from the root)? Intuitively, shallower feels more ancestor-like.

FIX► No — there is a real trap here. **Depth and discovery order are not the same.** Counterexample: the root has a left and a right subtree, and DFS finishes the **whole left subtree** before going right. Then a **deep** vertex in the left subtree may have a **smaller** discovery number than a **shallow** vertex in the right subtree — because it was visited first. We want "who was discovered first" (global time order), not "who is shallower" (local structure). **Use dfn (discovery order); depth is wrong.** This matters because low's whole logic rests on "reaching back along time order"; swap time order for depth and the "stack dfn is increasing" property (§ 6) fails, collapsing the entire closure mechanism.

4 • The three-edge struggle (core part I): which info is usable

This section is the most error-prone part of Tarjan, and the one most worth struggling through slowly. We know low must "propagate up the subtree and take min," but concretely — when processing each out-edge ($u \rightarrow j$) of u , exactly what information of j do we use to update $\text{low}[u]$? I will lay out the real trial-and-error step by step.

4.1 How cycles form, and the picture of "merging"

First, see the cycle clearly. Along a tree path $\text{root} \rightarrow \dots \rightarrow a$, if a has an edge back to an earlier ancestor on the path, a cycle closes; all its vertices are mutually reachable, hence one class. Further — **cycles merge through a shared vertex**: a closes one cycle, and another vertex on it has an edge reaching an even earlier ancestor, so the two cycles fuse into a larger SCC, pushing the true root up to that earliest ancestor. This is precisely why low must **propagate up the subtree and take min**: how far a substructure can reach is reported to the parent, the parent to the grandparent... until some vertex finds "my subtree at best reaches only myself" — it is the root.

4.2 The first instinct, and its counterexample

Q ▶ Simple, then: for every edge $u \rightarrow j$, just do $\text{low}[u] = \min(\text{low}[u], \text{low}[j])$? However far j can reach back, u via j can too.

A ▶ It holds for "unvisited tree-edge children" — after recursing into j , $\text{low}[j]$ has converged, safe to use. But for an "already-visited j "? You assumed "the early vertex j can reach, u via this edge can also reach." That is **not always true**.

Q ▶ Why not always? j is right there, isn't $\text{low}[j]$ the earliest it can return to?

FIX► Counterexample: suppose j belongs to another subtree that has **already been fully explored and recognized as a separate SCC**. Such a j has a valid old $\text{low}[j]$, yes, but its component is **already closed — there is no "live" cycle between it and u** . Pulling $\text{low}[j]$ into $\text{low}[u]$ now means treating an early vertex that **cannot form a cycle with u** as "reachable back by u " — u 's low is polluted, and two components that should stay separate get wrongly merged.

So "use $\text{low}[j]$ regardless of visited status" is wrong. The key is not "visited or not," but "is j 's component closed."

4.3 A key intermediate discovery: $\text{low}[j]$ need not point to an ancestor — it may point to a "sibling"

Q► Wait, I assumed $\text{low}[j]$ always points to an ancestor of j (or u). Is that so?

A► No — a discovery worth pausing on. Via non-tree edges, **$\text{low}[j]$ can point to a non-direct-ancestor, e.g. a "sibling" in another subtree** (reached by a cross edge). This splits "visited j " into two essentially different cases:

- the sibling's subtree is **still active** (not yet closed) $\Rightarrow u$ may still loop back through it to a common ancestor (the LCA or higher) and form a cycle $\Rightarrow \text{low}[j]$ is valid;
- the sibling's subtree is **already closed** $\Rightarrow$ no live path between it and u , no cycle $\Rightarrow \text{low}[j]$ is invalid.

The "ancestor vs sibling" distinction is the root of "visited j must split into two further cases."

4.4 Nailing the test: closure, not visitation

The test (the core result of this derivation): when computing $\text{low}[u]$, for a neighbor j , **only if j 's component is "not yet closed" do we execute $\text{low}[u] = \min(\text{low}[u], \text{low}[j])$; skip the closed ones.** The criterion is "**is j 's vertex closed,**" independent of whether $\text{vis}[j]$ is true — because "visited" still splits into "closed" and "not closed."

Q► Can I save effort and judge "is j closed" purely from the numeric value of $\text{low}[j]$? Say, $\text{low}[j]$ below some level means closed?

FIX ► No — the last hole to plug. **A closed vertex' s $low[j]$ is still a valid small number, indistinguishable from an active vertex' s $low[j]$ by value alone.** By the number you **cannot tell apart** "an active vertex that can still climb back" from "a closed cross-target." So "closed or not" is an **independent state** of j that must be **recorded separately**; it cannot be inferred from low' s value. — This "extra record," we will find in § 6, is exactly "is j still on the stack."

5 • dfn[j] or low[j]: the most subtle fork

§ 4 said: for an "already-visited, not-closed" j we may update $\text{low}[u]$. But with **which quantity of j — $\text{dfn}[j]$ or $\text{low}[j]$** ? This is the subtlest spot in implementing Tarjan, the one most people gloss over. Let us settle it completely.

Q► For a back edge $u \rightarrow j$ (j an ancestor still on the stack), should I write $\text{low}[u] = \min(\text{low}[u], \text{low}[j])$ or $\text{low}[u] = \min(\text{low}[u], \text{dfn}[j])$? Since $\text{low}[j]$ carries more information (it may be even smaller than $\text{dfn}[j]$), isn't $\text{low}[j]$ better — it propagates further?

A► First, see what the edge actually guarantees. The edge $u \rightarrow j$ only guarantees "u can step to vertex j ." It does **not** guarantee "u can reach the farther vertex that j 's subtree can reach (where $\text{low}[j]$ points)." That farther vertex is reachable by **j itself**, not necessarily by **u via this one edge**.

A► Next: if u and j are in fact in the same still-growing large cycle, the dfn of that "earlier vertex" **will naturally converge to u via another path (the tree-edge chain)** — no need for this back edge to carry $\text{low}[j]$. Hence — **for back edges / non-tree edges to active vertices, $\text{dfn}[j]$ suffices and is always safe; $\text{low}[j]$ is safe only when j is genuinely still on the stack (not closed)**.

FIX► Why is $\text{dfn}[j]$ "always safe" while $\text{low}[j]$ is "conditional"? Because **$\text{dfn}[j]$, once fixed at j 's first visit, never changes**; whereas **$\text{low}[j]$ is dynamic and may not have converged** — a j still mid-recursion, exploration unfinished, may have a merely intermediate $\text{low}[j]$ that will still be pulled down. Updating others with a "still-changing intermediate value" is dangerous. $\text{dfn}[j]$ is a fixed anchor, so standard implementations universally choose $\text{dfn}[j]$.

Conclusion: (1) tree-edge child (recursion returned) $\Rightarrow$ use **low[j]** (converged, trustworthy); (2) back edge / non-tree edge to an active vertex $\Rightarrow$ use **dfn[j]** (fixed anchor, always safe). The two are equivalent when j is on the stack, but the dfn[j] version is harder to get wrong.

This note's companion code uses an equivalent compact form: a uniform `if (!processed[low[j]]) low[r]=min(low[r],low[j])`. It is also correct because (a) `low[r]=dfn[r]` is set on entering DFS, (b) the `low[j]` of a not-closed j converges correctly via the tree-edge chain when it and r share a cycle, and (c) a closed j is blocked by processed. Only after understanding the dfn-vs-low fork can you see why this compact form holds.

Q► Then "what if `low[j] < dfn[j]`" — isn't discarding `low[j]` a loss?

A► No loss. `low[j] < dfn[j]` means j can still reach back to something earlier than itself; but if that "earlier vertex" truly shares a cycle with u, its dfn reaches u via the **tree-edge path** on the cycle, not via this back edge. The back edge need only honestly contribute "I reached vertex j (`dfn[j]`)," one certain fact. Handing the uncertain `low[j]` upstream may instead cause a wrong merge when j is already closed. **Being conservative (using `dfn[j]`) is exactly correctness here, not a compromise.**

6 • The root test and the stack: extracting a component cleanly

6.1 The root test $\text{low}[u] == \text{dfn}[u]$

On backtracking from u , after all its out-edges are processed, if $\text{low}[u] == \text{dfn}[u]$, it means " u and its subtree can at best reach back to u itself, nothing earlier" — so u is the **root** of its SCC (the minimum-dfn vertex of the component), and we close it now.

This test holds uniformly for singleton and multi-vertex components, no special case. An isolated vertex (or one with no back edges) also satisfies $\text{low} == \text{dfn}$, and on closing pops only itself, a size-1 SCC. Remember "zero-step self-reachability" from 1.1? It cashes out here: a single vertex is a legitimate equivalence class. **Anyone who wants a separate if-branch for singletons has not thought the test through.**

6.2 Using a stack to extract a component: why a stack, and pop until where

Q► $\text{low}[u] == \text{dfn}[u]$ holds, I know u is a root — but "which vertices does this component contain"? How to take them all out at once?

A► Use a stack. Rule: **push each vertex on first visit**; when $\text{low}[u] == \text{dfn}[u]$, **pop continuously from the top** until u itself is popped — the popped segment (including u) is exactly all members of u 's SCC.

Q► Why does "pop from top until u " carve out exactly this component, no more no less? Could it pull in vertices of another component?

A► By two properties:

I **Stack entries have strictly increasing dfn** (bottom to top) — pushed in visit order, earlier-visited have smaller dfn, nearer the bottom.

II **After an SCC pops, the remaining stack still has increasing dfn** — popping only shaves a contiguous segment off the top, not breaking the order below.

From these: at $\text{low}[u] == \text{dfn}[u]$, all stack vertices with $\text{dfn} \geq \text{dfn}[u]$ form exactly a **contiguous top segment**, with u at its bottom (smallest dfn). And **sub-components closed earlier have already left the stack**, not on it at all, impossible to mis-grab. So "pop until u " (equivalently "pop all $\text{dfn} \geq \text{dfn}[u]$ ") carves out exactly u 's whole component, not one more, not one less.

FIX► A small detail in the pop condition: when testing " $\text{dfn}[x] \geq \text{dfn}[u]$," **the equality must not be dropped** — drop it and the root u itself ($\text{dfn}[x] == \text{dfn}[u]$) never pops, the component loses its root. An equivalent, less error-prone form is "pop until the popped vertex $== u$." Both carve the same segment, because increasing stack dfn guarantees the $\geq \text{dfn}[u]$ vertices are the contiguous top.

The stack does double duty — this is Tarjan's elegance: (1) it **extracts** each SCC's members; (2) it simultaneously implements the "closed flag" from § 4 — **"is neighbor j still on the stack" exactly equals "is j 's component closed"**. A vertex leaving the stack is equivalent to its component being closed. So the "extra closed state that must be recorded," sought painfully in § 4.4, is answered by "on the stack or not," at no extra space (implementations usually keep an in-stack boolean for $O(1)$ lookup).

7 • Correctness: chaining every judgment with no gaps

By now correctness has, in fact, been established piece by piece throughout the derivation. We gather it into a formal chain, each link pointing back to the corresponding section:

1. **An SCC is a mutual-reachability equivalence class**; V splits into disjoint maximal blocks. (§ 1.2–1.3, from $\sim$ being an equivalence relation)
2. **Each SCC has a unique root in the DFS tree** (its minimum-dfn vertex), and all its vertices form a **contiguous** stack segment. (§ 6.2, from "increasing stack dfn + popping preserves it")
3. **The semantics of $\text{low}[u]$ is fixed** as: the earliest dfn reachable from u and its subtree via edges whose component is "not closed." (§ 3–§ 4, forced out by the "capping" need)
4. **The update rule is correct**: min only over "not-closed" neighbors, skip closed; use $\text{dfn}[j]$ for back edges, $\text{low}[j]$ for returned tree children. A closed j forms no live cycle with u , so its info is invalid; a cycle that should merge has its earliest dfn converge to the root via the tree-edge chain. (§ 4.4, § 5)
5. **The root test $\text{low}[u] == \text{dfn}[u]$ is necessary and sufficient**: u reaches nothing earlier $\Leftrightarrow u$ is a component root $\Leftrightarrow$ safe to close now. Uniform for singletons. (§ 6.1)
6. **Completeness and mutual exclusivity of closure**: at $\text{low}[u] == \text{dfn}[u]$, the contiguous stack segment with $\text{dfn} \geq \text{dfn}[u]$ is exactly u 's whole SCC, not more (sub-components left earlier), not less (equality includes the root). Each vertex is closed exactly once. (§ 6.2)

Together these constitute the correctness proof of Tarjan's algorithm: it does place each vertex into, and only into, the equivalence class it truly belongs to, in full agreement with the pure definition of § 1. Every step is justified; none relies on "the textbook says so."

Complexity: each vertex is pushed once and popped once; each edge is processed exactly once in its source's loop. Hence time **$O(V+E)$** , space $O(V)$ (dfn, low, stack, in-stack flag). We have met the goal set in § 2: compress the naive $O(V^4)$ to linear, done as a by-product of **one DFS**.

8 • The gap from proof to code (core part II): bug-by-bug

Between "understanding the algorithm" and "writing correct code" there is always a gap. Below I lay out **every pit I actually fell into** — without exception, each is the consequence of "some derivation not landing correctly in code." This section is worth more than any "reference implementation," because it tells you **where the error will be, and why**.

8.1 Pit 1: low' s initial value, and a misplaced "fallback"

Q► I initialized all low to INT_MAX, then after the loop wrote `if(low[r]==INT_MAX) low[r]=dfn[r]` as a fallback — what is wrong?

FIX► Two errors.

(1) **Logic error:** "set to dfn[r] only if nobody updated it" — but the correct semantics is that low[r] should equal dfn[r] from the start (every vertex can at least reach itself), then be pulled down by neighbors via min, not "fallback if untouched."

(2) **Placement error (fatal):** the fallback is **after** the for-loop. But a back edge needs to read the ancestor' s low **during** the loop — and at that moment the ancestor' s low is still INT_MAX (its DFS has not returned, the post-loop fallback has not run). So on a back edge $u \rightarrow \text{ancestor}$, `min(low[u], INT_MAX)` does nothing, and **the back-edge information is blocked by your own code**. Symptom: all cycles fall apart into singletons. **Fix:** on the **first line** of DFS write `low[r] = dfn[r]`. Then any read of any vertex' s low is a valid value, and back edges min correctly. This echoes § 3' s semantics: "I can at least reach myself" is low' s **floor**, then pulled down step by step — low is a dynamically converging quantity, and its floor must be present from the very start.

8.2 Pit 2: the index coordinate of processed — node id or dfn number?

Q► I wrote `processed[low[j]]` to test whether j is closed, and `processed[dfn[x]]`=true on closing. It segfaulted — why?

A► On the surface, out of bounds: `low[j]` is `INT_MAX` before `j` gets a valid value, and `processed[INT_MAX]` overflows. The root fix is 8.1 — with the initial value fixed, `low` always lands in `[0,n)`.

One layer deeper is **coordinate consistency**: is `processed` indexed by "node id" or by "dfn number"? **Either works, but you must use one throughout.** This note's code chooses **indexing by dfn**: read `processed[low[j]]` (`low` stores a dfn value), write `processed[dfn[x]]`, both in dfn coordinates, self-consistent. If one place uses node id and another dfn, you "mark A but query B," and the logic falls apart. **dfn and node are in bijection, but bijection does not mean the indices are interchangeable — they are two numbering systems.**

8.3 Pit 3: querying the wrong subject in the test

FIX► A subtle semantic error: the test is "is `j`'s vertex closed," not "is the vertex corresponding to the numeric value `low[j]` closed." Under dfn indexing with unified `low/dfn` coordinates, `processed[low[j]]` happens to query "the closure state of the representative vertex `j` can reach," which under this algorithm's invariant agrees with "`j`'s component's closure state," so it holds. But if your mind swapped the subject to some unrelated vertex (e.g. an irrelevant cross-target), you would write code that queries the wrong object. **Before writing any array index, ask: whose state do I want right now?**

8.4 Pit 4: forgetting to pop in the closing loop

FIX► On closing, the while loop reads `st.top()`, marks `processed`, pushes into `scc` — but **forgets `st.pop()`**. The stack never shrinks, leading to an infinite loop or repeatedly collecting the same vertices. Pure slip, but extremely common. **Any loop that "takes something off the top and consumes it" must pair a pop right there.**

8.5 Pit 5: missing the equality in the pop test / using the wrong quantity

FIX► As § 6.2 said: with `dfn[x] ≥ dfn[r]`, **the equality must not be dropped**, or the root never pops. A safer form is "pop until `x==r`." Also do not write the pop test by comparing `low` (it should be `dfn`) — `low` changes, and using it as a pop boundary carves the wrong range.

8.6 A general self-check discipline

The only reliable way to translate "understanding" into "correctness": after writing, hand-trace one minimal triggering case. For SCC, the minimal case is the three-cycle $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$. Walk it line by line: when DFS reaches vertex 2 and it points back to 0, ask yourself — what is `low[0]` right now? Is it a valid value or `INT_MAX`? Which vertex does `processed[low[0]]` query? Should it update? — this single trace catches most of the five pits above within 30 seconds. **Real bugs are usually not "couldn't think of it" but "thought of it, then assumed I'd code it right and skipped verification."**

9 • Full code + seven test cases

Below is the complete runnable implementation (C++17), passing seven cases: single cycle, acyclic chain, cycle-with-tail, two components, shared-vertex merge into one large SCC, self-loop + isolated, larger mixed. Each landing point has a corresponding derivation above: set `low[r]=dfn[r]` on entry (§ 8.1); processed indexed by dfn, min only over not-closed neighbors (§ 4.4 / § 8.2); on closing, pop the contiguous `dfn[x] ≥ dfn[r]` segment with pop (§ 6.2 / § 8.4).

```
#include <limits>
#include <vector>
#include <stack>
#include <functional>
#include <algorithm>
using namespace std;

vector<vector<int>> tarjanSCC(int n, const vector<vector<int>>& g) {
    vector<int> dfn(n, -1), low(n, INT_MAX);
    vector<bool> vis(n, false), processed(n, false);
    stack<int> st;
    vector<vector<int>> ret;

    function<void(int,int)> dfs = [&](int r, int& cnt) {
        dfn[r] = cnt++;
        low[r] = dfn[r];          // KEY: set on entry; every node can reach itself
        st.push(r);
        for (int idx = 0; idx < (int)g[r].size(); ++idx) {
            int j = g[r][idx];
            if (!vis[j]) {        // tree edge: recurse first
                vis[j] = true;
                dfs(j, cnt);
            }
            if (!processed[low[j]]) // use low[j] only if j's component is NOT closed
                low[r] = min(low[r], low[j]);
        }
        if (low[r] == dfn[r]) {   // r is the root of an SCC -> close it
            vector<int> scc;
            while (!st.empty()) {
                int x = st.top();
                if (dfn[x] >= dfn[r]) {
                    processed[dfn[x]] = true; // mark closed (indexed by dfn)
                    scc.push_back(x);
                    st.pop();
                } else break;
            }
            ret.push_back(scc);
        }
    };

    int cnt = 0;
    for (int i = 0; i < n; ++i)
        if (!vis[i]) { vis[i] = true; dfs(i, cnt); }
    return ret;
}
```

The seven cases cover all boundaries: pure cycles verify merging; acyclic chains verify "each vertex is its own component"; cycle-with-tail verifies component boundaries; two-components and shared-vertex verify "merge vs no-merge"; self-loop + isolated verify the uniformity of the singleton test; larger-mixed is a combined regression. All PASS, exit 0.

10 • Where it came from: the Turing Award and the 1972 paper

The linear SCC algorithm comes from Robert Tarjan's 1972 paper "Depth-first search and linear graph algorithms." Its real contribution is not just "computing SCCs" but **pioneering a paradigm**: with **one DFS plus one cleverly designed auxiliary quantity** (our low, i.e. low-link), it compresses a whole class of graph problems — seemingly requiring multiple passes — into linear time. Linear-time SCC is only its most famous application; the same paper also covers biconnected components and more.

Tarjan and John Hopcroft jointly received the **1986 Turing Award** "for fundamental achievements in the design and analysis of algorithms and data structures." SCC / low-link is a flagship member of that work. Tarjan's signature move is consistent throughout: **one DFS plus one clever auxiliary, compressing multiple passes into linear** — dominator trees, offline LCA, and the near-constant amortized analysis of union-find are all incarnations of the same idea. Understanding this one low, you have already touched the pulse of his whole body of work.

11 • Coda: Euler's bridges, modeling > solving, first principles, manuscripts

Having come this far, it is worth stepping back to methodology — because it explains "why this note bothers to derive forward," and why classical discrete algorithms, though their conclusions are clean, deserve to be chewed over.

11.1 There are not many discrete-algorithm motifs

A reassuring fact: classical discrete algorithms are **finite, exhaustible, mutually reducible** — not a bottomless pit. The truly independent "motifs" number about a dozen: the graph-traversal family (DFS/BFS opens everything: SCC, topological sort, bridges and cut vertices, bipartite testing), the shortest-path family, the flow-and-matching family (max-flow is the master, bipartite matching its special case), greedy and matroids, DP, DSU/balanced trees/heaps. See the **reductions** among them and the whole map converges. The SCC mastered here is exactly a graph-traversal motif that uses DFS + one auxiliary to break open a whole region; tackle max-flow and bipartite matching with the same forward derivation and you will find they too grow from one core (the augmenting path). This contrasts sharply with continuous mathematics, where every new structure tends to grow a whole new machine.

11.2 Euler's bridges: modeling > solving

A common misconception is that "classical discrete algorithms are deep." But the scarce thing is never the clean final statement — it is the move of **daring to throw away everything irrelevant and keep only the structure**. Euler himself found the bridges problem nearly trivial — in a letter to the mayor of Danzig he said the solution "is based only on reason, and depends on no mathematical principle." Yet "trivial" is **a posteriori**: it is anchored **after** the object "graph" has taken shape. The step of **abstracting a real map into "vertices + edges"** — discarding bridge shape, length, geography, keeping only connectivity — took humanity nearly another 150 years to do routinely. The hard part is not the parity-of-degrees criterion (which falls out fairly naturally once the abstraction is in place) — it is **inventing that coordinate system itself**.

The correspondence record bears this out: Ehler formally asked Euler for a solution on 9 March 1736; by 13 March, to Marinoni, Euler already announced he had "found a proof that works not only for this case but for all problems of this kind"; he only replied to Ehler on 3

April. And he had reported the problem to the St. Petersburg Academy back in August 1735 — so the "solution" was long in hand, and the weeks-long gap was reply cadence, not solving time. **Modeling > solving: get the abstraction right and the solution is rarely far.** This note's repeated emphasis on "pin the problem first (§ 1), let the naive method point the way (§ 2)" is exactly this principle at the algorithmic level.

11.3 First principles = a borrowed library of axioms

The "first principles" you invoke as axioms are not axioms at all — they are **theorems** others proved and that you have compressed into intuition; you simply no longer pay the cost of deriving them. This is the strength of first-principles thinking (speed), and its risk: if a layer you trust hides a flaw, the whole tower tilts while you are none the wiser, because you long ago stopped checking it. So the truly strong move is to **periodically poke down one layer** and ask: "this thing I treat as an axiom — is it a theorem, or just a habit?"

This note did exactly that — it poked down one layer. Most people memorize low as an axiom; here we re-forced it out from the need to "judge component capping." That an axiom, once removed, can be rebuilt from deeper layers shows the knowledge is not a free-floating flashcard but a **connected web** — and that is the real line between "truly understanding" and "having memorized." Not panicking when an interviewer probes to the bottom relies on this web, not on that card.

11.4 Manuscripts are the source code

One last point, about this note's form itself. A formal paper **compresses** months of messy thought into a few pages of linear narrative — the reader sees a clean "theorem → proof → conclusion" chain, while the author's real path (tried A, failed, detoured to B, found a link to C, scrapped it, restarted) lives only in the **manuscript**. The clean version is lossy compression; the manuscript is the source code. This note deliberately preserved many "Q/A/FIX" struggles, cleaning a real "manuscript" into "annotated source code" rather than producing yet another clean version with no provenance. Its value lies not in final tidiness but in **preserving the crookedness of discovery** — because that crooked road is the only path that turns "knowing" into "owning."

Problem • forcing out low • the three-edge struggle • the dfn-vs-low fork • the root test and the stack • proof
• the proof-to-code gap • history — one complete forward derivation.